

The choice of activation function corresponds to the implemented real-valued logic; Gödel, product, and Łukasiewicz logic are common examples.

In addition to matching their corresponding connectives' truth functions, as described in Section 6, LNN requires monotonicity. Specifically, the activation functions for conjunction and disjunction must increase monotonically with respect to each operand, and the activation function for implication must decrease monotonically with respect to the antecedent and increase monotonically with respect to the consequent. In addition, it is useful though not required for $\otimes$ and real-valued disjunction $x \oplus y$ to be related via the De Morgan laws, for both operations to be commutative and associative, and for real-valued implication $x \rightarrow y$ to be the *residuum* of $\otimes$, or specifically $(x \rightarrow y) = \max\{z \mid y \geq (x \otimes z)\}$. These properties do not guarantee that $(x \rightarrow y) = (\neg x \oplus y)$, that $((x \rightarrow 0) \rightarrow 0) = x$, or that $(x \otimes x) = (x \oplus x) = x$, though they may individually hold for certain activation functions.

Gradient-based neural learning also requires differentiable parameters that can be tuned to improve model performance. We introduce the concept of *importance weighting*, whereby neural inputs with larger weight have more influence on neural output. This can take many forms, as thoroughly explored in [7], though this paper focuses on an easily computable weighting scheme based on nonlinear functions applied to dot products.

3.2 Weighted nonlinear logic

We introduce weighted generalizations of the standard real-valued logics in section B of the supplementary material. This completes the mapping of NNs to weighted real-valued logics. Here is shown a weighted generalization of Łukasiewicz-like logics. The n -ary *weighted nonlinear conjunctions*, used for logical AND, are given

$$\beta(\bigotimes_{i \in I} x_i^{\otimes w_i}) = f(\beta - \sum_{i \in I} w_i(1 - x_i)) \quad (1)$$

for $f : \mathbb{R} \rightarrow [0, 1]$ with $f(1 - x) = 1 - f(x)$, input set I , bias term $\beta \geq 0$, weights $w_i \geq 0$, and inputs $x_i \in [0, 1]$. The n -ary *weighted nonlinear disjunctions*, used for logical OR, are then

$$\beta(\bigoplus_{i \in I} x_i^{\oplus w_i}) = f(1 - \beta + \sum_{i \in I} w_i x_i). \quad (2)$$

Observe that β and the various w_i establish a hyperplane with respect to the inputs x_i , though clamped to the $[0, 1]$ range. For $f(x) = \max\{0, \min\{1, x\}\}$, the resulting activation functions are similar to the rectified linear unit (ReLU) from neural network literature and to the Łukasiewicz t- and s-norms. Alternate choices of f include the logistic function as shown in Figure 1b and a linearly interpolated *tailored activation function* as shown in Figure 1c, as further explored in Section 6.

Bias term β permits classically equivalent formulae $x \rightarrow y$, $\neg y \rightarrow \neg x$, and $\neg x \oplus y$ to be made equivalent in weighted nonlinear logic by adjusting β . The *weighted nonlinear residuum*, used for logical implication, is given by the residuum² of $\otimes$,

$$\beta(x^{\otimes w_x} \rightarrow y^{\oplus w_y}) = f(1 - \beta + w_x(1 - x) + w_y y). \quad (3)$$

Note the use of $\otimes$ in the antecedent weight but $\oplus$ in the consequent weight, which is meant to indicate the antecedent has AND-like weighting (scaling its distance from 1) while the consequent has OR-like weighting (scaling its distance from 0). This residuum is most disjunction-like when $\beta = 1$, most $(x \rightarrow y)$ -like when $\beta = w_y$, and most $(\neg y \rightarrow \neg x)$ -like when $\beta = w_x$.

3.3 Activation functions for atoms

Neurons pertaining to atoms require activation functions that aggregate truth values found for the various computations identified as proofs of the atom. For example, each of $(x_1 \otimes x_2 \otimes x_3) \rightarrow y$, $(x_1 \otimes x_4) \rightarrow y$, and $(x_2 \otimes x_4) \rightarrow \neg y$ may constitute proofs (or disproofs) of y . A typical means of aggregating proven truth values is to return the maximum proven lower bound and minimum proven upper bound. On the other hand, it may be desirable to employ importance weighting via weighted nonlinear logic in aggregation as well, substituting OR for max in the lower bound computation and AND for min in the upper bound. A natural choice of weights for such aggregations is the weights of the atoms as they occur in the formulae serving as their proofs. For example, if $x_1^{\otimes 3} \rightarrow y^{\oplus 2}$ and $x_2^{\otimes 1} \rightarrow y^{\oplus .5}$ are proofs of y , then aggregation would use weights 2 and .5, respectively.

²This solution assumes $f(x) = \max\{0, \min\{1, x\}\}$, but for simplicity may be used with any f .

4 Inference

Inference refers to the entire process of computing truth value bounds for (sub)formulae and atoms based on initial knowledge, ultimately resulting in predictions made at neurons pertaining to queried formulae or other results of interest. LNN achieves this with multiple passes over the represented formulae, propagating tightened truth value bounds from neuron to neuron until computation necessarily converges. The previous section already suggests the *upward pass* of inference, whereby formulae compute their truth value bounds based on bounds available for their subformulae. This section further describes the *downward pass*, which permits prior belief in the truth or falsity of formulae to inform truth value bound for propositions or predicates used in said formulae.

4.1 Bidirectional inference

In addition to the usual evaluation of connectives, LNN infers truth values bounds for each of a connective's inputs based on bounds on its output and other inputs. Depending on the type of connective involved, such computations correspond to the familiar inference rules of classical logic:

$$\begin{array}{ll} x, x \rightarrow y \vdash y & (\text{modus ponens}) \\ \neg y, x \rightarrow y \vdash \neg x & (\text{modus tollens}) \end{array} \quad \begin{array}{ll} x, \neg(x \wedge y) \vdash \neg y & (\text{conjunctive syllogism}) \\ \neg x, x \vee y \vdash y & (\text{disjunctive syllogism}) \end{array}$$

The precise nature of these computations depends on the selected family of activation functions. For example, as noted in Section 3.1, if implication is defined as the residuum, then *modus ponens* is performed via the logic's t-norm, i.e. AND. The remaining inference rules follow a similar pattern as prescribed by the functional or logical inverses of their upward computations.

The application of these inference rules immediately suggests a means of generating proofs for atoms based on the formulae they occur in. As further discussed in Section 4.3, given truth value bounds for a formula, it is possible to apply inference rules to obtain truth value bounds for each of its leaves.

4.2 Inference rules in weighted nonlinear logic

In the following, L and U denote lower and upper bounds found for neurons corresponding to the formulae indicated in their subscripts, e.g. $L_{x \rightarrow y}$ is the lower-bound truth value for the formula $x \rightarrow y$ as a whole while U_x is the upper bound for just x . The bounds computations for $\neg$ are trivial:

$$\begin{array}{ll} L_{\neg x} \geq \neg U_x = 1 - U_x, & L_x \geq \neg U_{\neg x} = 1 - U_{\neg x}, \\ U_{\neg x} \leq \neg L_x = 1 - L_x, & U_x \leq \neg L_{\neg x} = 1 - L_{\neg x}. \end{array}$$

The use of inequalities here acknowledges that tighter bounds for each value may be available from other sources. For instance, both y and $x \rightarrow \neg y$ can yield $L_{\neg y}$; the tighter of the two would apply.

Observing that, in weighted nonlinear logic, $\beta(x^{\otimes w_x} \rightarrow y^{\oplus w_y}) = \beta((1-x)^{\oplus w_x} \oplus y^{\oplus w_y})$ and $\beta(\bigotimes_{i \in I} x_i^{\otimes w_i}) = 1 - \beta(\bigoplus_{i \in I} (1-x_i)^{\oplus w_i})$, it is only necessary to define one set of inference rule computations to cover all connectives. The upward bounds computations for $\beta(\bigoplus_{i \in I} x_i^{\oplus w_i})$ are

$$L_{\bigoplus_i x_i} \geq \beta(\bigoplus_{i \in I} L_{x_i}^{\oplus w_i}), \quad U_{\bigoplus_i x_i} \leq \beta(\bigoplus_{i \in I} U_{x_i}^{\oplus w_i})$$

while the downward bounds computations for disjunction are

$$L_{x_i} \geq \beta^{w_i}((\bigotimes_{j \neq i} (1 - U_{x_j})^{\otimes w_j/w_i}) \otimes L_{\bigoplus_i x_i}^{\otimes 1/w_i}) \quad \text{if } L_{\bigoplus_i x_i} > 1 - \alpha, \quad \text{else } 0 \quad (4)$$

$$U_{x_i} \leq \beta^{w_i}((\bigotimes_{j \neq i} (1 - L_{x_j})^{\otimes w_j/w_i}) \otimes U_{\bigoplus_i x_i}^{\otimes 1/w_i}) \quad \text{if } U_{\bigoplus_i x_i} < \alpha, \quad \text{else } 1 \quad (5)$$

where α is threshold determined by f to address potential divergent behavior at $L_{\bigoplus_i x_i} \leq 1 - \alpha$ and $U_{\bigoplus_i x_i} \geq \alpha$. To understand why this occurs, observe that, for $f(x) = \max\{0, \min\{1, x\}\}$, i.e. the ReLU case, $x \oplus y$ can return 1 for many different values of x and y ; specifically, whenever $w_x x + w_y y \geq \beta$. Accordingly, if $U_{\bigoplus_i x_i} = 1$, we cannot infer an upper bound for any x_i .

4.3 The Upward-Downward algorithm

Inference propagates truth value bounds from neuron to neuron along in alternating upwards and downwards passes over the syntax trees of the represented formulae. The upward pass, shown in

Algorithm 1: Upward pass to infer formula truth value bounds from subformulae bounds

```
function upwardPass(formula z):  
  for operand  $x_i$  of  $z$ ,  $i \in I$  do                    # propagate bounds upward from leaves  
    | upwardPass( $x_i$ )  
  if  $z = \neg x$  then                                     # negation  
    | aggregate( $z$ ,  $(1 - U_x, 1 - L_x)$ )  
  else if  $z = \beta(\bigoplus_{i \in I} x_i^{\oplus w_i})$  then          # multi-input disjunction  
    | aggregate( $z$ ,  $(\beta(\bigoplus_{i \in I} L_{x_i}^{\oplus w_i}), \beta(\bigoplus_{i \in I} U_{x_i}^{\oplus w_i}))$ )  
  # Other operations are handled as combinations of the above  
function aggregate(formula  $z$ ,  $(L'_z, U'_z)$ ):  
  |  $(L_z, U_z) := (\max\{L_z, L'_z\}, \min\{U_z, U'_z\})$  # tighten existing bounds
```

Algorithm 2: Downward pass to infer subformula truth value bounds from formula bounds

```
function downwardPass(formula z):  
  for operand  $x_j$  of  $z$ ,  $j \in I$  do  
    if  $z = \neg x$  then                                     # negation  
      | aggregate( $x$ ,  $(1 - U_z, 1 - L_z)$ )  
    else if  $z = \beta(\bigoplus_{i \in I} x_i^{\oplus w_i})$  then          # multi-input disjunction  
      |  $L'_{x_j} := \beta/w_j((\bigotimes_{i \neq j} (1 - U_{x_i})^{\otimes w_i/w_j}) \otimes L_{\bigoplus_i x_i}^{\otimes 1/w_j})$  if  $L_{\bigoplus_i x_i} > 1 - \alpha$ , else 0  
      |  $U'_{x_j} := \beta/w_j((\bigotimes_{i \neq j} (1 - L_{x_i})^{\otimes w_i/w_j}) \otimes U_{\bigoplus_i x_i}^{\otimes 1/w_j})$  if  $U_{\bigoplus_i x_i} < \alpha$ , else 1  
      | aggregate( $x_j$ ,  $(L'_{x_j}, U'_{x_j})$ )  
    | downwardPass( $x_j$ )                                # propagate bounds downward to leaves
```

Algorithm 1, uses truth value bounds available at atoms to compute bounds at each subformula according to the normal evaluation of connectives based on their operands. The downward pass, shown in Algorithm 2, uses truth value bounds known for formulae and previously computed at other subformulae to tighten bounds at each subformula (and ultimately each atom) according to the inference rules given above. This process repeats, as shown in Algorithm 3, until convergence, as proved in section C of the supplementary material:

Theorem 1. *Given monotonic $\neg$, $\oplus$, and f , Algorithm 3 converges to within ϵ in finite time.*

5 LNN bounds as probability bounds

This section presents a variant of LNN where lower and upper bound truth values at each subformula serve as bounds on the probability that the subformula is True in classical logic. This is achieved by using different activation functions for lower and upper bound computations. Again observing that implication and conjunction may be defined in terms of negation and disjunction, these are

$$\begin{aligned} L_{\bigoplus_i x_i} &\geq \max_{i \in I} L_{x_i} & L_{x_i} &\geq \max\{0, L_{\bigoplus_i x_i} - \sum_{j \neq i} U_{x_j}\}, \\ U_{\bigoplus_i x_i} &\leq \min\{1, \sum_{i \in I} U_{x_i}\} & U_{x_i} &\leq U_{\bigoplus_i x_i} \end{aligned}$$

with negation unchanged from above. Let A be the set of atomic formulae and let $g : A \rightarrow \{T, F\}$ be an interpretation. Further define $g(\sigma)$ for any formula σ on A to be the truth value of σ under the truth-value assignments by g to the atomic formulae. Let Λ be the set of all interpretations.

A sentence is an expression of the form (σ, l, u) where σ is a formula and $l, u \in [0, 1]$. A theory is a set of sentences $\Gamma = \{(\sigma_1, l_1, u_1), \dots, (\sigma_k, l_k, u_k)\}$. Define $S_\sigma \triangleq \{g \mid g(\sigma) = T\}$ for any formula

Algorithm 3: Recurrent inference procedure with recursive directional graph traversal

```
function inference(formulae z):  
  while  $\sum(|\delta L_z| + |\delta U_z|) > \epsilon$  do                # loop until convergence  
    for  $r \in \text{roots}(z)$  do                               # visit all formula roots in sequence  
      | upwardPass( $r$ )                                     # leaves-to-root traversal  
      | downwardPass( $r$ )                                   # root-to-leaves traversal
```
